

CNN Assignment

Giovanny Rosales

October 2026

can you generate 3 essay questions about deep neural networks, loss functions, and convolutional neural networks as a way for me to study?



Absolutely. Here are three essay-style questions designed to make you connect concepts rather than just define them:

1. Deep Neural Networks:

Explain how a deep neural network learns to approximate a complex relationship between inputs and outputs. In your answer, discuss the roles of layers, neurons, weights, biases, activation functions, forward propagation, and backpropagation. Why can increasing the number of hidden layers make a network more powerful, and what problems can arise when a network becomes very deep?

2. Loss Functions:

Describe the role of a loss function during neural-network training. Explain how the loss function, gradient descent, and backpropagation work together to update the network's weights. Compare **mean squared error** and **cross-entropy loss**, including the types of problems for which each is typically used. What would happen if an inappropriate loss function were chosen for a particular problem?

3. Convolutional Neural Networks:

Explain how a convolutional neural network (CNN) processes an image differently from a traditional fully connected neural network. Discuss kernels/filters, convolution, feature maps, stride, padding, pooling, and learned weights. Explain why CNNs are particularly effective for images and how the features learned in early convolutional layers typically differ from those learned in deeper layers.

A good study strategy would be to answer each in **about 1–2 paragraphs without notes**, then I can grade your answers like an exam and point out anything you're missing.

1.) A deep neural network learns to approximate relationships between inputs and outputs through layers of neurons. The first layer is known as the input layer, taking in raw data as inputs. The following layer(s) consist of hidden neurons that taken in a linear combination of inputs, random weights and biases. New weights and biases are generated and this continues until the output layer. In order to learn more complex trends an activation function is used, that will set some rule on the generated linear combination. The final

output layer is then incorporated into a loss function that determines how well the network is doing. The network weights are updated via backpropagation gradient descent and forward propagated to update the loss function. This process continues until the loss has flattened to an acceptable value near zero.

2.) The loss function is responsible for keeping your network from overfitting and underfitting. It does this by mathematically stepping towards or away from the mean in some statistically chosen way. When the loss is high, the network undergoes gradient descent, determine the rate of change for all input parameters affecting the output and determining the direction for which the loss function is minimized. The mean squared error is a loss function that determines the squared distance from the mean of a dataset while the cross-entropy loss using logarithmic penalties to minimize the loss. An incorrect loss function would not properly explain the output and lead to overfitting/underfitting.

3.) Convolutional neural networks process images by sharing the weights for a distribution of inputs that are used to compute hidden layers. The kernel defines the size of the shared weights. The stride determines how the starting weight is shifted to the next. The padding determines what happens at the edges of the image. The pooling determines how many weights are skipped in your kernel. CNNs are effective for images because it reduces the overall computational cost of predictions by sharing weights.

Updated Answers

1.) A deep neural network learns to approximate relationships between inputs and outputs through layers of neurons. The first layer is known as the input layer, taking in raw data as inputs with initialized weights and biases. The following layer(s) consist of hidden neurons that take in a linear combination of their own inputs, weights and biases. In order to learn more complex trends an activation function is used, that will set some rule on the generated linear combination. The activation function introduces nonlinearity which makes it possible for the network to learn complex nonlinear relationships. Without an activation function, the network would simply be a linear transformation. This is all known as the forward pass which will determine predictions. A final loss function is then used to compare predictions with true targets. Backpropagation computes gradients of that loss with respect to the parameters. The optimizer will update the parameters to minimize the loss.

2.) The loss functions determines how wrong the model's predictions are. Backpropagation computes the gradients while gradient descent uses those gradients to update the weights. Mean squared error measures the squared difference between the predicted value and the true value, mainly used for regression. Cross entropy loss is used for classification problems, where the network predicts probabilities for classes. An incorrect loss function can lead to inefficient

learning.

3.) Convolutional neural networks process images by sharing the weights for a distribution of inputs that are used to compute hidden layers. The kernel defines the size and shape of the shared weight kernel. The stride determines how the starting kernel is shifted to the next. The padding determines what happens at the edges of the image. Padding can add or set values to zero for your kernel near the edge of an image. The pooling operates on the feature maps which reduces spatial dimension. CNNs are effective for images because they reduce the overall computational cost of predictions by sharing weights.

Gap Finding

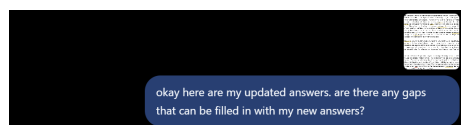


Figure 1: Prompt for gaps.

The gaps it found seemed to be theoretically driven with explanations being more rooted with the actual mathematics.

What have I learned?

I think proper usage of LLMs like this assignment is a very good way to learn and to learn quicker. Proper prompting is only becoming more mandatory in the sciences. Of course, simply copying everything is not the correct thing to be doing. You should be critically thinking and analyzing the answers any LLM is giving you. Cross referencing the responses or questions will make it even better.

Compliance

I acknowledge the use of ChatGPT for this assignment. Everything is kept within this assignment.

Yes — these are **substantially better**. You fixed almost all of the major conceptual errors from the first version. What remains is mostly filling in missing pieces and tightening a few definitions rather than relearning anything.

1. Deep neural networks

This is now your strongest answer. I'd put it around 8.5/10.

One small technical correction: the **input layer itself doesn't usually have weights and biases**. The weights and biases belong to the connections/neurons in the next layer. So instead of:

"taking in raw data as inputs with initialized weights and biases"

I would say:

"taking in the raw input features, which are passed to the first hidden layer through initialized weights and biases."

Your explanation of activation functions is much better now. You correctly identified the crucial reason for them:

"The activation function introduces nonlinearity..."

That's exactly what I'd want to see on an exam.

You are still missing one part of my original question: **why deeper networks can cause problems**. Add something like this at the end:

Increasing the number of hidden layers allows the network to learn increasingly complex and hierarchical representations, but very deep networks can suffer from problems such as overfitting, vanishing or exploding gradients, and increased computational cost.

Also, you can explicitly include the bias in the neuron equation if you want to strengthen the answer:

(a)

Also, you can explicitly include the bias in the neuron equation if you want to strengthen the answer:

$$z = \sum_i w_i x_i + b_i \quad a = f(z).$$

That demonstrates you understand exactly what a neuron is doing.

2. Loss functions

Huge improvement. This went from your weakest response to a pretty solid one. I'd say 8/10.

This sentence is now correct:

"The loss function determines how wrong the model's predictions are."

And this distinction is especially good:

"Backpropagation computes the gradients while gradient descent uses those gradients to update the weights."

Keep that. That's an important distinction professors love asking about.

There are two things I'd add.

First, mention the **learning rate**. Gradient descent doesn't just follow the gradient; it determines the size of the parameter update through something like

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w},$$

where η is the learning rate.

Second, say a little more about cross-entropy. Right now you correctly say it is used for classification, but you can explain why:

(b)

There are two things I'd add.

First, mention the **learning rate**. Gradient descent doesn't just follow the gradient; it determines the size of the parameter update through something like

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w},$$

where η is the learning rate.

Second, say a little more about cross-entropy. Right now you correctly say it is used for classification, but you can explain why:

Cross-entropy uses logarithmic penalties to compare predicted class probabilities with the true class and strongly penalizes predictions that are confidently incorrect.

That's enough—you don't necessarily need the whole formula unless your professor expects equations.

I'd also change your last sentence slightly. Instead of just:

"An incorrect loss function can lead to inefficient learning."

Use:

An inappropriate loss function can cause the network to optimize an objective that does not properly represent the task, resulting in poor or inefficient learning.

That's more precise.

(c)

3. Convolutional neural networks

This one still needs the most work, but your pooling and padding descriptions are much better. I'd put it around 7/10 now.

The first sentence is still a little vague:

"CNNs process images by sharing the weights for a distribution of inputs..."

I'd replace that with something much more concrete:

A CNN processes an image using small learned filters called kernels that slide across local regions of the image. The same kernel weights are reused at different spatial locations, a process known as weight sharing.

That's really the central idea of convolution.

For kernel size, I'd change your wording to:

The kernel size determines the spatial dimensions of the filter, such as 3×3 or 5×5 , and therefore determines the size of the local region examined at one time.

For stride, your idea is right, but I'd make it:

The stride determines how many pixels the kernel moves each time it slides across the image.

For padding, what you've written is basically correct. A stronger version is:

Padding adds values, typically zeros, around the edges of an image so that edge information can be processed and the dimensions of the resulting feature map can be controlled.

Your pooling definition is now correct:

"pooling operates on the feature maps which reduces spatial dimension."

(d)

You could just add an example:

For example, max pooling retains the largest activation within a small region while reducing the spatial dimensions of the feature map.

The biggest missing CNN concept: feature maps

You mention them but don't define them. Add:

When a kernel is applied across an image, it produces a feature map that indicates where the pattern represented by that kernel is detected.

For instance, imagine one kernel has learned to detect vertical edges. Its feature map would have high activation in areas of the image containing vertical edges.

And you're still missing the **early layers vs. deep layers** part of the original question. This is important:

Earlier convolutional layers typically learn simple features such as edges, lines, and color gradients. Deeper layers combine these simpler features into increasingly complex structures such as textures, shapes, and eventually object-level features.

Think of it as

pixels → edges → textures → shapes → objects

Finally, CNNs aren't useful only because they're computationally cheaper. That's part of it, but I'd say:

CNNs are effective for images because they exploit spatial structure through local connections and weight sharing. This dramatically reduces the number of parameters compared with a fully connected network while allowing the network to detect the same features regardless of where they occur in an image.

That's a much stronger explanation.

(e)

Finally, CNNs aren't useful only because they're computationally cheaper. That's part of it, but I'd say:

CNNs are effective for images because they exploit spatial structure through local connections and weight sharing. This dramatically reduces the number of parameters compared with a fully connected network while allowing the network to detect the same features regardless of where they occur in an image.

That's a much stronger explanation.

Where you stand now

Your conceptual understanding has improved quite a bit:

Topic	First attempt	Updated
Deep neural networks	~7/10	~8.5/10
Loss functions	~4.5/10	~8/10
CNNs	~5.5/10	~7/10

The main things I'd study now aren't basic definitions anymore. I'd focus on these relationships:

input → $Wx + b$ → activation → prediction → loss → backpropagation → o

and for CNNs:

image → kernel → convolution → feature map → activation → pooling.

You're at the point where another round of essay questions that are **slightly harder and more application-based** would probably be more useful than repeating these same three.

(f)